

CampusGuard: Final Design & Work Split

COS 214 Practical 5 · Team reference · Due Tuesday 29 September 2026

How to use this document. This replaces the earlier *Design Approach Overview*. It keeps that design (same story, same six patterns, same class names wherever possible) and fixes the points that would have cost marks. Section 9 is the work split. The appendix contains the **agreed header files**: they already compile under `-std=c++11 -Wall -Wextra -pedantic` and are in the accompanying starter zip. Once they are pushed, all three of us code against them in parallel. **Changing a shared header needs a heads-up in the group chat first.**

1. What changed from the original design, and why

Problem in original	Why it matters	Fix
Facade described as the <i>only</i> front door	Brief 5.4: subsystem operations must “remain independently usable”	Facade stays the easy entry point; Scenario 2 drives the console and commands directly
Only 3 response units	Brief 6: “at least four collaborating response components”	Added CommsTeam as a 4th mediator colleague
No failure / invalid-operation case	Explicit functional requirement	Named state events; illegal ones throw InvalidTransition , the console reports REJECTED. PA offline and unknown area are also handled
IncidentState had one <i>handle()</i>	Linear stepping means nothing can ever be invalid	dispatch() / startWork() / resolve() , each legal only in certain states
AccessControlSystem never defined	Receiver of LockAreaCommand; can’t draw or code it	Defined as a modern subsystem owned by CampusGuardSystem
Command had only execute(); CancelCommand pointed at another command	Nothing to reverse; dangling-pointer risk into console history	undo() added to Command; cancelling is OperatorConsole::cancelLast() . Added EvacuateAreaCommand (brief lists evacuation) so there are still 4 commands
Legacy PA call left vague	Brief 5.3: must translate, not just wrap	Fixed legacy API: int zones, 1-5 priority, C-string max 60 chars, int error codes
No ownership policy; facade pointed at one Incident	Rule 4, and we need ≥ 2 incidents	CampusGuardSystem owns everything long-lived in a fixed order (section 5)
All “holds a list” drawn as hollow diamonds	Owning lists are composition, not aggregation	Filled diamond where the holder owns; see the UML notation table
Mediator had one generic notify()	Tends to become a large if/else chain (rule 8)	Three specific methods. Each unit decides respondsTo() polymorphically
Only one scenario described	Task 3 needs two, each with ≥ 2 patterns	Two scenarios, section 6
No Makefile / Docker / Valgrind / GDB / Git / PDF plan	Task 5 and the submission	Assigned in the split

2. The system in one paragraph

CampusGuard coordinates a campus incident from report to resolution. An incident is opened and moves through **Reported** → **Dispatched** → **InProgress** → **Resolved** (State). Every status change is pushed to a live **OperatorDashboard** and an audit **IncidentLogger** (Observer). Operator actions (dispatch a unit, lock an area, issue an alert, evacuate, cancel) are objects run by the **OperatorConsole** (Command). When a unit is dispatched, the **EmergencyCoordinator** decides which other units help (Mediator). Alerts go out over the old campus PA through **LegacyPAAdapter** (Adapter). **EmergencyResponseFacade** packages the common multi-step workflows (Facade).

3. GoF participant mapping (exact C++ class names)

Pattern	Participants	Owner
Command	Command: <i>Command</i> · ConcreteCommands: <i>DispatchUnitCommand</i> , <i>LockAreaCommand</i> , <i>IssueAlertCommand</i> , <i>EvacuateAreaCommand</i> · Invoker: <i>OperatorConsole</i> · Receivers: <i>ResponseUnit</i> , <i>Incident</i> , <i>AccessControlSystem</i> , <i>ExternalAlertService</i> · Client: <i>EmergencyResponseFacade</i> / scenario code	B
Adapter	Target: <i>ExternalAlertService</i> · Adaptee: <i>LegacyCampusPASystem</i> · Adapter: <i>LegacyPAAAdapter</i> (object adapter) · Clients: <i>IssueAlertCommand</i> , <i>EvacuateAreaCommand</i> , <i>CommsTeam</i>	B
Mediator	Mediator: <i>EmergencyMediator</i> · ConcreteMediator: <i>EmergencyCoordinator</i> · Colleague: <i>ResponseUnit</i> · ConcreteColleagues: <i>SecurityTeam</i> , <i>MedicalResponder</i> , <i>FacilitiesCrew</i> , <i>CommsTeam</i>	C
Facade	Facade: <i>EmergencyResponseFacade</i> · Subsystems: <i>OperatorConsole</i> (+ commands), <i>Incident</i> , <i>AccessControlSystem</i> , <i>ExternalAlertService</i> , and via dispatch the mediator network	C
State (extra)	Context: <i>Incident</i> · State: <i>IncidentState</i> · ConcreteStates: <i>ReportedState</i> , <i>DispatchedState</i> , <i>InProgressState</i> , <i>ResolvedState</i>	A
Observer (extra)	Subject: <i>Incident</i> · Observer: <i>IncidentObserver</i> · ConcreteObservers: <i>OperatorDashboard</i> , <i>IncidentLogger</i>	A

Incident legitimately plays two roles (State context and Observer subject): its status is what changes, and its status is what others watch. Rule 7 allows this because the design genuinely justifies it. **Do not add any seventh pattern** by accident (no Singleton, no Template Method base class, no Factory class): the brief says exactly two extras.

Why State and Observer (for the PDF justification)

	Problem it solves	Better than the simpler alternative because...	Where it appears
State	Which operations are legal depends on the incident's stage	An enum + switch in every command spreads the rules around and is banned by rule 8. Each state class owns its own rules; adding a stage is one new class	<i>DispatchUnitCommand</i> calls <i>incident.dispatch()</i> ; facade calls <i>startWork()/resolve()</i> ; illegal events give the failure case
Observer	Several parts must react to a status change without the incident knowing them	Hard-coding dashboard/logger calls into <i>Incident</i> couples it to every display. Polling wastes work and misses changes. New watchers attach without touching <i>Incident</i>	Every transition in both scenarios updates the dashboard and log

Observer vs Mediator (a likely tutor question): Mediator is *two-way coordination between peers* (units act, the coordinator decides who else acts). Observer is a *one-way broadcast* from one subject to passive watchers that never affect the incident. Different problems, so there is no overlap.

4. Behaviour rules everyone codes to

4.1 Incident lifecycle (State)

Current state	dispatch()	startWork()	resolve()
ReportedState	→ DispatchedState	InvalidTransition	InvalidTransition
DispatchedState	stays (extra unit)	→ InProgressState	InvalidTransition
InProgressState	stays (extra unit)	InvalidTransition	→ ResolvedState
ResolvedState	InvalidTransition	InvalidTransition	InvalidTransition

Inside a state method, `incident.changeState(...)` must be the **last statement**, because it deletes the state object that is running. `changeState` saves the old name first, swaps `state_`, then notifies observers.

4.2 Coordination (Mediator)

- A unit that is dispatched, recalled or asks for support tells the coordinator, and never talks to another unit directly.

- The coordinator's whole rule: for every **other** registered unit, if respondsTo(incident), call assist(incident, reason). There is no type switch in the coordinator.
- respondsTo: Security = Intrusion, Fire · Medical = Medical, Fire, Hazmat · Facilities = Fire, Hazmat (assist opens the area for responders via AccessControlSystem) · Comms = severity ≥ Moderate (assist makes a PA announcement through ExternalAlertService, which is the adapter).
- **assist() must never call back into the mediator**, otherwise you get infinite ping-pong.

4.3 Legacy PA translation (Adapter)

CampusGuard wants (ExternalAlertService)	Legacy PA understands (fixed)	Adapter does
area as a string, e.g. "CHEM"	int zone code	look up in mapArea() table; unknown gives false
Severity Low / Moderate / Critical	int priority 1-5	Low→1, Moderate→3, Critical→5
std::string of any length	const char*, max 60 chars	split into ≤ 60-char chunks, one PA_Broadcast each
bool delivered	0 / -1 / -2 / -3 / -4	0 → true; otherwise print the reason and return false

4.4 Facade workflows (each runs ≥ 3 subsystem operations, stops at the first failure)

Operation	Steps in order
respondToIncident(incident, lead)	1. submit DispatchUnitCommand(lead) (State + Mediator) · 2. submit IssueAlertCommand (Adapter) · 3. incident.startWork()
declareLockdown(incident)	1. submit DispatchUnitCommand(security) · 2. submit LockAreaCommand(Locked) · 3. submit IssueAlertCommand(lockdown notice)
standDown(incident)	1. submit LockAreaCommand(Unlocked) · 2. incident.resolve() · 3. submit IssueAlertCommand(all clear)

4.5 Runtime output convention

Every printed line starts with a tag so the tutor can follow the collaboration: [Console] [Command] [State] [Dashboard] [Logger] [Coordinator] [Security] [Medical] [Facilities] [Comms] [Adapter] [LegacyPA] [Access] [Facade]. Print a banner line at the start of each scenario and each facade operation.

5. Ownership policy and UML notation

One composition root: CampusGuardSystem owns every long-lived object as members, declared in this order. C++ destroys members in **reverse** order, so everything that holds a reference dies before the thing it refers to:

legacyPA_ → alertService_ → accessControl_ → dashboard_, logger_ → coordinator_ → units_ → incidents_ → console_ (owns commands) → facade_

- **Composition (unique_ptr):** CampusGuardSystem → units, incidents, facade · OperatorConsole → Command history · Incident → current IncidentState.
- **Non-owning (reference / raw pointer):** commands → receivers · units → mediator, AccessControlSystem, ExternalAlertService · coordinator → units · Incident → observers · adapter → legacy PA · facade → subsystems.
- Observers are declared before incidents, so they outlive them: no dangling observer pointers and no need to detach at shutdown.
- facade_ is a unique_ptr created **last in the constructor body**, because it needs the security unit, which only exists once units_ has been filled.
- Every polymorphic base has a virtual destructor (already in the headers). The Incident destructor is defined in Incident.cpp (unique_ptr to an incomplete type).
- C++11 has no std::make_unique: use std::unique_ptr<T>(new T(...)).

Relationship	UML symbol	Examples
Inherits from a class	solid line, hollow triangle	DispatchedState → IncidentState, SecurityTeam → ResponseUnit
Implements an interface	dashed line, hollow triangle (or solid line; be consistent)	LegacyPAAAdapter → ExternalAlertService
Owns (lifetime tied)	filled diamond at the owner, with multiplicity	OperatorConsole ◆— 0..* Command; Incident ◆— 1 IncidentState
Holds, but does not own	hollow diamond or plain association arrow	EmergencyCoordinator ◇— 0..* ResponseUnit; Incident — 0..* IncidentObserver
Uses one object	plain arrow	LockAreaCommand → AccessControlSystem

Class names, method names and arrows in every diagram must match the C++ exactly. The tutor may ask any of us to trace a UML message to its method.

6. The two end-to-end scenarios

Scenario 1: Chemistry lab fire (Member C)	Scenario 2: Library intrusion at night (Member B)
Incident #1: Fire, Critical, area CHEM. Runs through the Facade .	Incident #2: Intrusion, Moderate, area LIB. Runs the console directly, with no facade .
1. openIncident: Dashboard + Logger see Reported 2. facade.respondToIncident(fire, medical) a. DispatchUnitCommand: State Reported→Dispatched, observers notified b. medical.dispatchTo → coordinator → Security assists (crowd control), Facilities opens CHEM, Comms makes a PA announcement via the Adapter c. IssueAlertCommand → LegacyPAAAdapter → PA priority 5, long text chunked d. startWork: Dispatched→InProgress 3. EvacuateAreaCommand for CHEM submitted directly 4. facade.standDown(fire): unlock, resolve, all-clear 5. dashboard.printBoard(), logger.printLog() Patterns: all six (Facade, Command, State, Observer, Mediator, Adapter)	1. openIncident 2. console.submit(DispatchUnitCommand(security)): State + Observer, and Comms assists via the mediator (Moderate) 3. console.submit(LockAreaCommand LIB Locked) 4. legacyPA_PA_SetAmplifier(LIB zone, 0), then IssueAlertCommand → adapter returns false → REJECTED , not recorded 5. incident.resolve() while Dispatched → InvalidTransition caught and printed 6. LockAreaCommand on unknown area "XYZ" → REJECTED 7. console.cancelLast() → undoes the LIB lock 8. startWork, resolve, then DispatchUnitCommand on the resolved incident → REJECTED Patterns: Command, State, Observer, Mediator, Adapter

Scenario code lives in src/ScenarioOne.cpp and src/ScenarioTwo.cpp as CampusGuardSystem member functions, so B and C never edit the same file.

7. Brief compliance checklist

Requirement (brief section)	Where it is satisfied
≥ 2 incidents with different data (6)	Fire/Critical/CHEM and Intrusion/Moderate/LIB
≥ 4 collaborating components (6)	Security, Medical, Facilities, Comms via the coordinator
≥ 3 concrete commands on real receivers (5.1, 6)	4 commands; each changes state, access or PA
Command triggers mediator coordination (6)	DispatchUnitCommand → dispatchTo → coordinator
Adapter translates an incompatible interface (5.3)	Section 4.3: 4 separate translations
Facade op with ≥ 3 subsystem ops; subsystems still usable (5.4)	Section 4.4; Scenario 2 and step 3 of Scenario 1 use subsystems directly
Both extra patterns in normal behaviour (6)	Every transition in both scenarios
Failure handled sensibly (6)	InvalidTransition, PA offline, unknown area
One scenario with ≥ 4 patterns (Task 3)	Scenario 1 has all six; Scenario 2 has five
No giant switch / type checks / client knowing every class (rule 8)	Polymorphic respondsTo; states own rules; enum names use lookup arrays
Virtual destructors; ownership policy (rules 3, 4)	In headers; section 5
-std=c++11 Makefile; Docker compose up --build (rules 2, 11)	In the starter zip

Requirement (brief section)	Where it is satisfied
4 diagrams (Task 4)	1 class, 2 sequence, 1 state machine (section 9)
Valgrind, GDB, Git history from all three (Task 5)	Assigned in section 9

8. Working independently

- **The headers are the contract.** One person pushes the starter zip to main on the day of the planning call. Everyone then implements only their own .cpp files.
- **Push these early, because others use them:** A pushes Incident + the states (used by B's DispatchUnitCommand and C's units); B pushes AccessControlSystem + LegacyPAAAdapter (used by C's units and facade). Target: the night of the call.
- **Test alone:** each person keeps a throwaway test file in tests/ (not in src/, so the Makefile ignores it). Fake interfaces where needed, e.g. C can test CommsTeam with a tiny FakeAlertService implementing ExternalAlertService.
- **Draw diagrams from the headers** immediately; adjust them after integration.
- **Git:** main is always buildable. Work on feature/<name>-<topic>, open a PR, and have someone else merge it. Commit small and often from day one: markers look at history spread over time from all three of us.
- **GDB bug must be real:** whoever hits a crash first keeps the backtrace and command history and sends it to C. Likely candidates: using `this` after changeState, or a wrong member order.

9. Work split

	Member A	Member B	Member C
Patterns	State, Observer	Command, Adapter	Mediator, Facade
Source files	Incident.cpp, IncidentState.cpp, Observers.cpp	Commands.cpp, OperatorConsole.cpp, AccessControlSystem.cpp, LegacyCampusPASystem.cpp, LegacyPAAAdapter.cpp	EmergencyCoordinator.cpp, ResponseUnits.cpp, Emergency ResponseFacade.cpp, CampusGuardSystem.cpp, main.cpp
Scenario	-	Scenario 2 (ScenarioTwo.cpp)	Scenario 1 (ScenarioOne.cpp) + integration lead
Diagrams	Final class diagram (merges everyone's parts) + state machine diagram for the incident lifecycle	Sequence 1: DispatchUnitCommand → State → Observer → Coordinator → units	Sequence 2: respondToIncident / declareLockdown → commands → Adapter → legacy PA
Engineering	Valgrind evidence (in Docker)	Makefile, Dockerfile, docker-compose.yml, README; fresh-clone Docker test	GDB investigation
PDF	Assembles the final PDF; writes the system description	Docker section + run instructions	Design & ownership rationale
Demo narration	Opening + State/Observer	Scenario 2	Scenario 1

Why it is fair: A has the simplest classes, so A takes the largest diagram (the merged class diagram) and PDF assembly. B has the most classes but they are straightforward, and Docker is mostly a one-time setup. C has the fewest classes but the hardest integration work (facade, wiring and main touch everyone's code).

Everyone

- Write the PDF sections for **your own patterns**: participant mapping, why it was used, and the ownership of your classes. Send them to A as text.
- Draw your own classes as a partial class diagram and send it to A for merging.
- Write your own contribution statement; help with the collaboration reflection.
- Be able to explain **all six** patterns, trace any command from invoker to receiver, and run docker compose up --build. The tutor can ask anyone.

Definition of done

Member A	Member B	Member C
Legal transitions change state and notify both observers; illegal ones throw with a clear message; dashboard board + log print correctly; no leaks	All 4 commands execute and undo; console rejects without recording; adapter translates all 4 ways and reports each error code; Docker builds from a fresh clone	Coordinator notifies only other responding units; all 3 facade ops stop on failure; wiring order matches section 5; both scenarios run end to end

10. Timeline

When	Goal
Sat 26 Sep (call)	Agree this doc, fill in names, push starter repo, everyone commits once. A and B push their early files that night
Sun 27 Sep	Every module implemented and tested alone. Diagrams drafted from headers. Docker working on the skeleton
Mon 28 Sep	Merge by midday (C leads); both scenarios running; Valgrind (A) and GDB (C) evidence; diagrams updated to match code; PDF sections to A by 18:00; full rehearsal in the evening, starting from docker compose up --build
Tue 29 Sep	Fresh-clone Docker check, final PDF + source archive, submit. Confirm the exact submission time.

11. Submission PDF order (brief section 8)

Team details → system description (A) → final class diagram (A) → participant mapping (all) → justification of State and Observer (A) → three behavioural diagrams (B, C, A) → design & ownership rationale (C) → GDB (C) and Valgrind (A) → Dockerfile, compose and run instructions (B) → GitHub link + collaboration reflection (all) → contribution statements (each).

12. Demo plan (5-7 minutes)

- 0:00 A runs `docker compose up --build` live so the tutor sees the build, and gives the one-paragraph system story.
- 1:00 C narrates Scenario 1 as it prints, pointing out each pattern as it appears.
- 3:30 B narrates Scenario 2: direct commands, three rejected operations, cancel.
- 5:30 A shows the dashboard board and the log, and states the ownership policy in one sentence.
- Then questions. Don't present one pattern at a time: tell the story.

Appendix: agreed header files (the contract)

Identical to include/ in the starter zip. Each file names its owner. They compile standalone with -std=c++11.

Types.h

```
#ifndef TYPES_H
#define TYPES_H
// SHARED (whole team). Plain data. Never branch on these to pick behaviour.
enum class IncidentType { Fire, Medical, Intrusion, Hazmat };
enum class Severity      { Low, Moderate, Critical };
enum class AccessMode    { Unlocked, Restricted, Locked };

inline const char* toString(IncidentType t) {
    static const char* n[] = { "Fire", "Medical", "Intrusion", "Hazmat" };
    return n[static_cast<int>(t)];
}
inline const char* toString(Severity s) {
    static const char* n[] = { "Low", "Moderate", "Critical" };
    return n[static_cast<int>(s)];
}
inline const char* toString(AccessMode m) {
    static const char* n[] = { "Unlocked", "Restricted", "Locked" };
    return n[static_cast<int>(m)];
}
#endif
```

Incident.h

```
#ifndef INCIDENT_H
#define INCIDENT_H
// OWNER: Member A | State (Context) + Observer (Subject)
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>
#include "Types.h"

class IncidentState;
class IncidentObserver;

class InvalidTransition : public std::logic_error {
public:
    explicit InvalidTransition(const std::string& msg) : std::logic_error(msg) {}
};

class Incident {
public:
    Incident(int id, IncidentType type, Severity severity,
            const std::string& area, const std::string& description);
    ~Incident(); // defined in .cpp (unique_ptr to incomplete type)
    Incident(const Incident&) = delete;
    Incident& operator=(const Incident&) = delete;

    int id() const { return id_; }
    IncidentType type() const { return type_; }
    Severity severity() const { return severity_; }
    const std::string& area() const { return area_; }
    const std::string& description() const { return description_; }
    std::string status() const; // current state's name()

    // Lifecycle events, delegated to the current state. Throw InvalidTransition if illegal.
    void dispatch();
    void startWork();
    void resolve();

    // non-owning. attach() immediately calls observer.update(*this, "", status())
    // so a new observer sees the current status. Observers must outlive the incident.
    void attach(IncidentObserver& observer);
    void detach(IncidentObserver& observer);

    // Called by IncidentState subclasses only. Save old name FIRST, then replace state_
    // (this deletes the calling state), then notifyObservers(old, new).
    void changeState(std::unique_ptr<IncidentState> next);

private:
    void notifyObservers(const std::string& oldStatus, const std::string& newStatus);

    int id_;
    IncidentType type_;
    Severity severity_;
    std::string area_;
    std::string description_;
    std::unique_ptr<IncidentState> state_; // owning
    std::vector<IncidentObserver*> observers_; // non-owning
};
#endif
```

IncidentState.h

```
#ifndef INCIDENT_STATE_H
#define INCIDENT_STATE_H
// OWNER: Member A | State (State + ConcreteStates)
// Reported --dispatch--> Dispatched --startWork--> InProgress --resolve--> Resolved
// dispatch() is also legal in Dispatched/InProgress (extra units; state unchanged).
// Every other event throws InvalidTransition. Resolved rejects everything.
// RULE: incident.changeState(...) must be the LAST statement (it deletes `this`).
#include <string>
#include "Incident.h"

class IncidentState {
public:
    virtual ~IncidentState() {}
    virtual std::string name() const = 0;
    virtual void dispatch(Incident& incident); // defaults throw InvalidTransition
    virtual void startWork(Incident& incident);
    virtual void resolve(Incident& incident);
};

class ReportedState : public IncidentState {
public:
    std::string name() const override;
    void dispatch(Incident& incident) override;
};

class DispatchedState : public IncidentState {
public:
    std::string name() const override;
    void dispatch(Incident& incident) override;
    void startWork(Incident& incident) override;
};

class InProgressState : public IncidentState {
public:
    std::string name() const override;
    void dispatch(Incident& incident) override;
    void resolve(Incident& incident) override;
};

class ResolvedState : public IncidentState {
public:
    std::string name() const override;
};

#endif
```

IncidentObserver.h

```
#ifndef INCIDENT_OBSERVER_H
#define INCIDENT_OBSERVER_H
// OWNER: Member A | Observer (Observer interface)
#include <string>
class Incident;

class IncidentObserver {
public:
    virtual ~IncidentObserver() {}
    virtual void update(const Incident& incident,
                       const std::string& oldStatus, const std::string& newStatus) = 0;
};

#endif
```

Observers.h

```
#ifndef OBSERVERS_H
#define OBSERVERS_H
// OWNER: Member A | Observer (ConcreteObservers). Both keep real data, not just print.
#include <cstdlib>
#include <map>
#include <string>
#include <vector>
#include "IncidentObserver.h"

class OperatorDashboard : public IncidentObserver {
public:
    void update(const Incident& incident,
               const std::string& oldStatus, const std::string& newStatus) override;
    void printBoard() const;           // live table: id, type, area, status
    std::size_t activeCount() const;   // incidents not yet Resolved
private:
    std::map<int, std::string> board_;   // id -> formatted row
    std::map<int, bool> resolved_;
};

class IncidentLogger : public IncidentObserver {
public:
    void update(const Incident& incident,
               const std::string& oldStatus, const std::string& newStatus) override;
    void printLog() const;             // numbered audit trail
    std::size_t entryCount() const { return entries_.size(); }
private:
    std::vector<std::string> entries_;
};
#endif
```

Command.h

```
#ifndef COMMAND_H
#define COMMAND_H
// OWNER: Member B | Command (Command interface)
#include <string>
class Command {
public:
    virtual ~Command() {}
    virtual void execute() = 0;         // throw std::exception on failure
    virtual void undo() = 0;
    virtual std::string describe() const = 0;
};
#endif
```

Commands.h

```
#ifndef COMMANDS_H
#define COMMANDS_H
// OWNER: Member B | Command (ConcreteCommands). Receivers are NON-OWNING references.
#include <string>
#include "Command.h"
#include "Types.h"

class Incident;
class ResponseUnit;
class AccessControlSystem;
class ExternalAlertService;

// execute: incident.dispatch() [State validates, may throw] then unit.dispatchTo(incident)
// undo:    unit.recall(incident)
class DispatchUnitCommand : public Command {
public:
    DispatchUnitCommand(ResponseUnit& unit, Incident& incident);
    void execute() override;
    void undo() override;
    std::string describe() const override;
private:
    ResponseUnit& unit_;
    Incident& incident_;
};

// execute: throws if area unknown; remembers previous mode; access.setMode(area, mode)
// undo:    restore previous mode
class LockAreaCommand : public Command {
public:
    LockAreaCommand(AccessControlSystem& access, const std::string& area, AccessMode mode);
    void execute() override;
    void undo() override;
    std::string describe() const override;
private:
    AccessControlSystem& access_;
    std::string area_;
    AccessMode mode_;
    AccessMode previous_;
};
```

```

};

// execute: alerts.sendAlert(...); throws std::runtime_error if it returns false
// undo: sends a Low-severity "CANCELLED: <message>" alert
class IssueAlertCommand : public Command {
public:
    IssueAlertCommand(ExternalAlertService& alerts, const std::string& area,
                      Severity severity, const std::string& message);
    void execute() override;
    void undo() override;
    std::string describe() const override;
private:
    ExternalAlertService& alerts_;
    std::string area_;
    Severity severity_;
    std::string message_;
};

// execute: unlock the area's exits, then Critical "EVACUATE <area>" alert (throws on failure)
// undo: restore previous access mode + "Evacuation cancelled" alert
class EvacuateAreaCommand : public Command {
public:
    EvacuateAreaCommand(AccessControlSystem& access, ExternalAlertService& alerts,
                        const std::string& area);
    void execute() override;
    void undo() override;
    std::string describe() const override;
private:
    AccessControlSystem& access_;
    ExternalAlertService& alerts_;
    std::string area_;
    AccessMode previous_;
};
#endif

```

OperatorConsole.h

```

#ifndef OPERATOR_CONSOLE_H
#define OPERATOR_CONSOLE_H
// OWNER: Member B | Command (Invoker)
#include <cstdint>
#include <memory>
#include <vector>
#include "Command.h"

class OperatorConsole {
public:
    // Owns + executes + records. Catches std::exception: prints "REJECTED: <why>",
    // does NOT record the command, returns false.
    bool submit(std::unique_ptr<Command> command);
    // Operator "cancel previous instruction": undo most recent command, remove it.
    bool cancelLast();
    std::size_t historySize() const { return history_.size(); }
private:
    std::vector<std::unique_ptr<Command>> history_; // owning
};
#endif

```

AccessControlSystem.h

```

#ifndef ACCESS_CONTROL_SYSTEM_H
#define ACCESS_CONTROL_SYSTEM_H
// OWNER: Member B | Modern CampusGuard subsystem. Receiver of LockArea/EvacuateArea.
#include <map>
#include <string>
#include "Types.h"

class AccessControlSystem {
public:
    void addArea(const std::string& area); // starts Unlocked
    bool hasArea(const std::string& area) const;
    bool setMode(const std::string& area, AccessMode mode); // false if unknown area
    AccessMode modeOf(const std::string& area) const; // std::out_of_range if unknown
private:
    std::map<std::string, AccessMode> modes_;
};
#endif

```

ExternalAlertService.h

```
#ifndef EXTERNAL_ALERT_SERVICE_H
#define EXTERNAL_ALERT_SERVICE_H
// OWNER: Member B | Adapter (Target): the interface CampusGuard WANTS.
#include <string>
#include "Types.h"

class ExternalAlertService {
public:
    virtual ~ExternalAlertService() {}
    virtual bool sendAlert(const std::string& area, Severity severity,
                          const std::string& message) = 0;    // true = delivered
};
#endif
```

LegacyCampusPASystem.h

```
#ifndef LEGACY_CAMPUS_PA_SYSTEM_H
#define LEGACY_CAMPUS_PA_SYSTEM_H
// OWNER: Member B | Adapter (Adaptee). FIXED legacy API: do not change these signatures.
// zones are ints, priority is 1..5 (5 = highest), text max 60 chars.
// PA_Broadcast returns 0 OK, -1 unknown zone, -2 amplifier offline,
// -3 bad priority, -4 text too long
#include <map>

class LegacyCampusPASystem {
public:
    void PA_RegisterZone(int zoneCode);           // amplifier starts ON
    void PA_SetAmplifier(int zoneCode, int on);   // demo hook: 0 = offline
    int PA_Broadcast(int zoneCode, int priority, const char* text);
private:
    std::map<int, int> amplifierOn_;
};
#endif
```

LegacyPAAdapter.h

```
#ifndef LEGACY_PA_ADAPTER_H
#define LEGACY_PA_ADAPTER_H
// OWNER: Member B | Adapter (Adapter). REAL translation:
// area string -> zone int, Severity -> priority (Low 1, Moderate 3, Critical 5),
// std::string -> const char* chunks of <= 60 chars, int return code -> bool + reason.
#include <map>
#include <string>
#include "ExternalAlertService.h"
#include "LegacyCampusPASystem.h"

class LegacyPAAdapter : public ExternalAlertService {
public:
    explicit LegacyPAAdapter(LegacyCampusPASystem& pa) : pa_(pa) {}
    void mapArea(const std::string& area, int zoneCode);
    bool sendAlert(const std::string& area, Severity severity,
                  const std::string& message) override;
private:
    LegacyCampusPASystem& pa_;           // non-owning
    std::map<std::string, int> zones_;
};
#endif
```

EmergencyMediator.h

```
#ifndef EMERGENCY_MEDIATOR_H
#define EMERGENCY_MEDIATOR_H
// OWNER: Member C | Mediator (Mediator interface)
#include <string>
class ResponseUnit;
class Incident;

class EmergencyMediator {
public:
    virtual ~EmergencyMediator() {}
    virtual void registerUnit(ResponseUnit& unit) = 0;
    virtual void unitDispatched(ResponseUnit& from, Incident& incident) = 0;
    virtual void unitRecalled(ResponseUnit& from, Incident& incident) = 0;
    virtual void supportRequested(ResponseUnit& from, Incident& incident,
                                 const std::string& reason) = 0;
};
#endif
```

ResponseUnit.h

```
#ifndef RESPONSE_UNIT_H
#define RESPONSE_UNIT_H
// OWNER: Member C | Mediator (Colleague). Knows ONLY the mediator, never other units.
// Also the receiver of DispatchUnitCommand.
#include <string>
#include "EmergencyMediator.h"
#include "Incident.h"

class ResponseUnit {
public:
    ResponseUnit(const std::string& name, EmergencyMediator& mediator)
        : name_(name), mediator_(&mediator) {}
    virtual ~ResponseUnit() {}
    const std::string& name() const { return name_; }

    virtual bool respondsTo(const Incident& incident) const = 0; // unit's own capability
    virtual void dispatchTo(Incident& incident) = 0; // do work, then announceDispatch()
    virtual void recall(Incident& incident) = 0; // do work, then announceRecall()
    // Mediator -> unit. MUST NOT call back into the mediator (no ping-pong).
    virtual void assist(Incident& incident, const std::string& reason) = 0;

    void requestSupport(Incident& incident, const std::string& reason) {
        mediator_>supportRequested(*this, incident, reason);
    }
protected:
    void announceDispatch(Incident& incident) { mediator_>unitDispatched(*this, incident); }
    void announceRecall(Incident& incident) { mediator_>unitRecalled(*this, incident); }
private:
    std::string name_;
    EmergencyMediator* mediator_; // non-owning
};
#endif
```

ResponseUnits.h

```
#ifndef RESPONSE_UNITS_H
#define RESPONSE_UNITS_H
// OWNER: Member C | Mediator (ConcreteColleagues) - the four collaborating components.
#include <string>
#include "ResponseUnit.h"

class AccessControlSystem;
class ExternalAlertService;

class SecurityTeam : public ResponseUnit {           // Intrusion, Fire
public:
    explicit SecurityTeam(EmergencyMediator& m) : ResponseUnit("Security", m) {}
    bool respondsTo(const Incident& incident) const override;
    void dispatchTo(Incident& incident) override;
    void recall(Incident& incident) override;
    void assist(Incident& incident, const std::string& reason) override;
};

class MedicalResponder : public ResponseUnit {       // Medical, Fire, Hazmat
public:
    explicit MedicalResponder(EmergencyMediator& m) : ResponseUnit("Medical", m) {}
    bool respondsTo(const Incident& incident) const override;
    void dispatchTo(Incident& incident) override;
    void recall(Incident& incident) override;
    void assist(Incident& incident, const std::string& reason) override;
};

class FacilitiesCrew : public ResponseUnit {         // Fire, Hazmat: opens access route
public:
    FacilitiesCrew(EmergencyMediator& m, AccessControlSystem& access)
        : ResponseUnit("Facilities", m), access_(access) {}
    bool respondsTo(const Incident& incident) const override;
    void dispatchTo(Incident& incident) override;
    void recall(Incident& incident) override;
    void assist(Incident& incident, const std::string& reason) override;
private:
    AccessControlSystem& access_;                     // non-owning
};

class CommsTeam : public ResponseUnit {             // severity >= Moderate: PA via adapter
public:
    CommsTeam(EmergencyMediator& m, ExternalAlertService& alerts)
        : ResponseUnit("Comms", m), alerts_(alerts) {}
    bool respondsTo(const Incident& incident) const override;
    void dispatchTo(Incident& incident) override;
    void recall(Incident& incident) override;
    void assist(Incident& incident, const std::string& reason) override;
private:
    ExternalAlertService& alerts_;                     // non-owning
};
#endif
```

EmergencyCoordinator.h

```
#ifndef EMERGENCY_COORDINATOR_H
#define EMERGENCY_COORDINATOR_H
// OWNER: Member C | Mediator (ConcreteMediator)
// Rule: for every OTHER registered unit u: if u->respondsTo(incident) -> u->assist(...)
#include <vector>
#include "EmergencyMediator.h"

class EmergencyCoordinator : public EmergencyMediator {
public:
    void registerUnit(ResponseUnit& unit) override;
    void unitDispatched(ResponseUnit& from, Incident& incident) override;
    void unitRecalled(ResponseUnit& from, Incident& incident) override;
    void supportRequested(ResponseUnit& from, Incident& incident,
                          const std::string& reason) override;
private:
    std::vector<ResponseUnit*> units_;                 // non-owning
};
#endif
```

EmergencyResponseFacade.h

```
#ifndef EMERGENCY_RESPONSE_FACADE_H
#define EMERGENCY_RESPONSE_FACADE_H
// OWNER: Member C | Facade. Each operation runs >= 3 subsystem steps in order and
// STOPS at the first failure (returns false). Subsystems stay directly usable.
#include "AccessControlSystem.h"
#include "ExternalAlertService.h"
#include "Incident.h"
#include "OperatorConsole.h"
#include "ResponseUnit.h"

class EmergencyResponseFacade {
public:
    EmergencyResponseFacade(OperatorConsole& console, AccessControlSystem& access,
                           ExternalAlertService& alerts, ResponseUnit& security)
        : console_(console), access_(access), alerts_(alerts), security_(security) {}

    // DispatchUnitCommand(lead) -> IssueAlertCommand -> incident.startWork()
    bool respondToIncident(Incident& incident, ResponseUnit& leadUnit);
    // DispatchUnitCommand(security) -> LockAreaCommand(Locked) -> IssueAlertCommand
    bool declareLockdown(Incident& incident);
    // LockAreaCommand(Unlocked) -> incident.resolve() -> IssueAlertCommand("all clear")
    bool standDown(Incident& incident);
private:
    OperatorConsole& console_;           // all non-owning
    AccessControlSystem& access_;
    ExternalAlertService& alerts_;
    ResponseUnit& security_;
};
#endif
```

CampusGuardSystem.h

```
#ifndef CAMPUS_GUARD_SYSTEM_H
#define CAMPUS_GUARD_SYSTEM_H
// OWNER: Member C (composition root). Owns EVERY long-lived object.
// Members die in REVERSE declaration order, so anything holding a reference is
// declared AFTER what it refers to. Do not reorder.
#include <memory>
#include <string>
#include <vector>
#include "AccessControlSystem.h"
#include "EmergencyCoordinator.h"
#include "EmergencyResponseFacade.h"
#include "Incident.h"
#include "LegacyCampusPASystem.h"
#include "LegacyPAAdapter.h"
#include "Observers.h"
#include "OperatorConsole.h"
#include "ResponseUnits.h"

class CampusGuardSystem {
public:
    CampusGuardSystem();           // wiring: areas, PA zones, units, registration
    void runScenarioOne();          // src/ScenarioOne.cpp (Member C)
    void runScenarioTwo();          // src/ScenarioTwo.cpp (Member B)
private:
    // creates, attaches dashboard + logger, stores; returns non-owning ref
    Incident& openIncident(IncidentType type, Severity severity,
                          const std::string& area, const std::string& description);

    LegacyCampusPASystem legacyPA_;
    LegacyPAAdapter alertService_;
    AccessControlSystem accessControl_;
    OperatorDashboard dashboard_;
    IncidentLogger logger_;
    EmergencyCoordinator coordinator_;
    std::vector<std::unique_ptr<ResponseUnit>> units_;
    SecurityTeam* security_;         // non-owning shortcuts into units_
    MedicalResponder* medical_;
    FacilitiesCrew* facilities_;
    CommsTeam* comms_;
    std::vector<std::unique_ptr<Incident>> incidents_;
    OperatorConsole console_;
    std::unique_ptr<EmergencyResponseFacade> facade_; // built LAST in ctor body (needs *security_)
    int nextIncidentId_;
};
#endif
```